

AI/ML DETECTION OF DNS TUNNELING & DATA EXFILTRATION

R-25-26J-066



Project Proposal Report

Sayuni Sahasna Dewapriya - IT22339256

B.Sc. (Hons) Degree in Information Technology

Department of Computer System Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

DECLARATION

I hereby declare that this is my own work, and that this proposal has not incorporated, without proper acknowledgment, any material previously submitted for a degree or diploma at any other university or institution of higher learning. To the best of my knowledge and belief, it does not contain any material previously published or written by any other person, except where explicit acknowledgment is made in the text.

Name	Student ID	Signature
Dewapriya S S	IT22339256	

The candidate is currently conducting research for the undergraduate dissertation under my supervision.

Supervisor Signature	
 Mr. Kavinga Yapa (Supervisor)	 Ms. Ayesha Wijesooriya (Co-Supervisor)
Date: 28/08/2025	Date: 28/08/2025

ABSTRACT

This project outlines the development of an AI/ML-driven system to detect DNS tunneling and data exfiltration, with a focus on enhancing real-time network security. The system employs advanced machine learning techniques to monitor both plaintext and encrypted DNS traffic (DoH/DoT) without requiring decryption, overcoming key limitations of current detection approaches. Using a hybrid methodology that blends supervised classification and anomaly detection, the system effectively identifies malicious activities such as DNS tunneling and data exfiltration, while ensuring minimal latency and high accuracy. The solution will be versatile, capable of running in both endpoint and SIEM-integrated environments, making it suitable for both small-scale tests and large enterprise deployments. A customizable rule engine will allow for flexible detection thresholds and dynamic response actions. Additionally, its compatibility with SIEM platforms ensures easy integration into existing security infrastructures. By prioritizing real-time detection and efficiency, the system aims to offer a scalable and adaptable defense against emerging DNS-based cyber threats.

TABLE OF CONTENTS

Declaration	1
Abstract	2
Table of Contents	3
List of Abbreviations	4
List of Figures	5
List of Tables	5
1. Introduction	6
1.1. Background	6
1.2. Literature Survey	7
1.3. Research Gap	9
1.4. Research Problem	12
2. Objectives	15
2.1. Main Objective	15
2.2. Specific Objectives	15
3. Methodology	17
3.1. Methodology Description	17
5.2. Tools & Technologies	31
5.3. Anticipated Conclusion	32
5.4. Work Breakdown Structure	33
5.5. Project Gantt Chart	33
6. Project Requirements	34
6.1. Functional Requirements	34
6.2. Non-Functional Requirements	34
6.3. User Requirements	35
References	36

LIST OF ABBREVIATIONS

- AI – Artificial Intelligence
- ML – Machine Learning
- DL – Deep Learning
- CNN – Convolutional Neural Network
- LSTM – Long Short-Term Memory
- RNN – Recurrent Neural Network
- SVM – Support Vector Machine
- RF – Random Forest
- KNN – K-Nearest Neighbors
- DNN – Deep Neural Network
- GRU – Gated Recurrent Unit
- DoH – DNS over HTTPS
- DoT – DNS over TLS
- DNS – Domain Name System
- IP – Internet Protocol
- C2 – Command and Control
- TLS – Transport Layer Security
- SIEM – Security Information and Event Management
- WHOIS – (Protocol for domain information lookup)
- NXDomain – Non-Existent Domain (DNS response code)
- PCA – Principal Component Analysis
- DBSCAN – Density-Based Spatial Clustering of Applications with Noise
- XGBoost – Extreme Gradient Boosting (ML algorithm)
- SHAP – SHapley Additive exPlanations (explainability technique)
- SOP – Standard Operating Procedure
- AUP – Acceptable Use Policy
- CIA – Confidentiality, Integrity, Availability
- IAAA – Identity, Authentication, Authorization, and Auditing
- ELK – Elasticsearch, Logstash, Kibana (SIEM stack)

LIST OF FIGURES

Figure 1:An Illustrative Example of DNS Tunneling Queries and Responses 6

Figure 2: System Architecture Diagram..... 20

Figure 3: Work Breakdown Structure 33

Figure 4: Project Gannt Chart 33

LIST OF TABLES

Table 1: Comparative Research Gap in Tabular View 13

Table 2:Tools & Technologies..... 31

1. INTRODUCTION

1.1. Background

The Domain Name System (DNS) is a core part of the Internet, translating human-readable domain names into IP addresses [1]. Because it is essential and trusted, DNS traffic is often overlooked in network security monitoring, making it an attractive target for attackers [2]. One major threat is DNS tunneling, which hides data inside DNS queries and responses to bypass firewalls, intrusion detection systems, and other security controls [3]. This technique is commonly used for data exfiltration, command-and-control (C2) communications, and maintaining persistence in compromised networks [4].

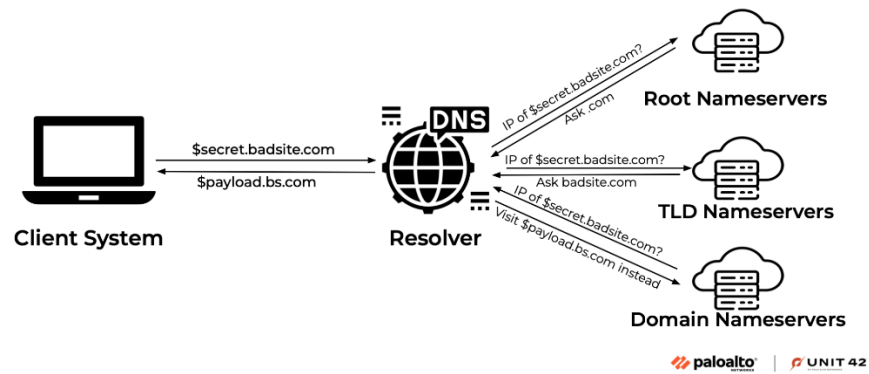


Figure 1: An Illustrative Example of DNS Tunneling Queries and Responses

DNS tunneling can be carried out in different ways, depending on the protocol used to transport the data. Examples include HTTP and HTTPS tunneling within web traffic, FTP tunneling during file transfers, and POP3 tunneling in email communications [1]. These techniques allow attackers to disguise malicious traffic as legitimate network activity, making detection more difficult.

The growing use of encrypted DNS protocols such as DNS over HTTPS (DoH) and DNS over TLS (DoT) has added another challenge. While these protocols improve privacy by encrypting DNS queries, they also provide attackers with an effective way to conceal malicious activity [5] [6]. Encrypted DNS traffic cannot be directly inspected due to privacy compliance requirements when handling customer data, making it difficult to analyze the content of the traffic. As a result, detection must focus on metadata such as packet sizes, timing patterns, and TLS handshake details [5] [6].

Researchers have explored many Machine Learning (ML) and deep learning (DL) methods for detecting DNS tunneling, including convolutional neural networks (CNN), long short-term memory (LSTM), recurrent neural networks (RNN), statistical models, support vector machines (SVM), and hybrid approaches [2] [3] [4] [6] [7] [8] [9] [10] [11]. Some studies have achieved over 99% accuracy in controlled settings, but most have focused solely on accuracy without addressing the challenges of real-time detection [4] [9] [12]. These methods are often evaluated in offline settings, which limits their application for live threat monitoring [1] [6]. To enable real-time detection, it is essential to also consider the recall rate, ensuring that no true positives are missed, as this is crucial for continuous threat visibility [3] [5].

This project proposes an AI/ML-based DNS threat detection system for near-real-time identification of DNS tunneling and data exfiltration, including malicious DoH/DoT traffic, without decryption. The system will support two modes: (1) endpoint-based detection for individual devices, primarily used for testing purposes during the initial stages of development, and (2) integration with SIEM platforms for enterprise-wide monitoring. A customizable rule engine will allow environment-specific policies and adaptive thresholds to reduce false positives, bridging the gap between academic research and practical deployment.

1.2.Literature Survey

A significant body of academic research has explored the use of AI and machine learning to detect DNS tunneling. Several detection approaches have been employed, each with varying degrees of complexity and accuracy.

1.2.1. Convolutional Neural Networks (CNN) + Clustering

Liang et al. proposed FECC, a CNN-based model combined with clustering techniques like k-means to detect unknown tunneling behaviors. This method is effective for identifying complex patterns in DNS traffic and can help distinguish between legitimate and tunneling traffic. The CNN model learns hierarchical features, while clustering groups similar traffic patterns for better identification [2].

1.2.2. Deep Learning Models (RNN, LSTM, DNN)

Zhang et al. compared 1D-CNN, RNN, and DNN models to detect DNS tunneling in real-world networks. Their results showed 99.90% accuracy, demonstrating the power of deep learning models in DNS tunneling detection [4]. Similarly, Sakarkar et al. used LSTM models, which outperformed GRU, 1D-CNN, and RNN, achieving 98.42% accuracy. These approaches excel at recognizing temporal patterns and relationships in DNS traffic, providing high detection accuracy [7].

1.2.3. Anomaly Detection (Unsupervised Learning)

Borges et al. built a cloud-focused detection model using statistical methods such as Poisson, Gaussian, and log-normal distributions to identify anomalies in DNS traffic [3]. This approach does not require labeled data and is particularly useful in dynamic environments like cloud platforms. Anomaly detection methods, like those explored by Chowdhary et al. and Li et al., are commonly used in detecting DNS tunneling by identifying deviations from expected behavior without the need for manually engineered features [10] [9].

1.2.4. Bigram Character Frequency Scoring

Qi et al. developed a bigram character frequency scoring method capable of identifying DNS tunneling patterns in near real-time. By analyzing the frequency of character pairs in domain names, this method can distinguish between normal and anomalous DNS requests. It is a statistical approach that requires minimal computational resources, making it suitable for real-time DNS traffic monitoring [6].

1.2.5. TLS Fingerprinting for DoH Tunneling

Encrypted DNS tunneling, specifically DNS over HTTPS (DoH), has also received attention in recent research. Zhan et al. showed that TLS fingerprinting could identify DoH-based exfiltration by analyzing the TLS handshake and traffic patterns. This approach is particularly effective in detecting DNS tunneling within encrypted traffic, where traditional methods may fail [5]. Nguyen et al. applied semi-supervised learning and statistical feature extraction to detect malicious DoH tunnels, achieving 98% accuracy in their detection [13].

1.2.6. Machine Learning with Feature Engineering

In several studies, researchers have combined traditional machine learning classifiers with entropy-based features and advanced feature engineering for DNS tunneling detection. For instance, Chowdhary et al. and Li et al. utilized features such as entropy of hostnames and cache misses, applying algorithms like SVM, random forests (RF), and K-nearest neighbors (KNN) to classify DNS traffic. These models rely heavily on manually engineered features, which are tailored to capture the characteristics of tunneling traffic [10] [14].

1.2.7. Hybrid Machine Learning Models

Hybrid models that combine multiple machine learning techniques have been shown to improve detection accuracy. Almusawi et al. proposed a multilabel support vector machine (SVM), which outperformed single SVM models in DNS tunneling detection [11]. Similarly, Chen et al. utilized hybrid deep learning models combining LSTM and GRU with attention mechanisms, achieving superior results in DNS tunneling detection compared to single models [12].

1.2.8. End-to-End Deep Learning Models (No Feature Engineering)

Some research, such as Chen and Li, has employed end-to-end deep learning models like Char-CNN that do not require explicit feature engineering. These models learn directly from raw DNS traffic, eliminating the need for manual feature extraction. This approach is advantageous as it can potentially detect tunneling behaviors without relying on predefined rules or features [12].

1.3. Research Gap

Although research on DNS tunneling detection has achieved high accuracy with various machine learning and deep learning methods, several challenges remain for real-world use.

1.3.1. Lack of Near-Real-Time Detection in Operational Environments

Many studies are conducted in offline or lab settings using pre-existing datasets, which do not align with the demands of operational networks. In these environments, threats must be detected and mitigated as soon as possible to minimize the impact of data loss.

Real-time detection is vital for operational networks to rapidly respond to DNS tunneling attempts and prevent further harm. Early detection and blocking of tunneling or exfiltration are essential to reducing their impact. The proposed solution fills this gap by offering near-real-time detection, a capability that many existing studies overlook.

1.3.2. Limited Coverage for Encrypted DNS (DoH/DoT) Threat Detection Without Decryption

Detecting DoH and DoT tunneling is particularly challenging due to the encrypted nature of these protocols. Traditional content inspection methods are ineffective because the traffic is encrypted. Although some research has focused on using TLS fingerprinting and metadata analysis to detect DoH/DoT-based exfiltration, these approaches still have limitations and often fail to handle traffic in privacy-sensitive environments. It is crucial to detect DoH/DoT tunneling without decryption, especially to comply with privacy regulations and uphold data security standards. The proposed solution addresses this by detecting tunneling without the need to decrypt traffic, making it suitable for privacy-conscious settings.

1.3.3. Lack of Integration with Security Tools and Workflows

Many DNS detection tools operate independently from Security Information and Event Management (SIEM) systems, making it challenging to link DNS anomalies with other security events and delaying incident response. The proposed solution will start with endpoint deployment for testing and validation in the early stages of development. After refining it, the solution will be fully integrated with SIEM systems and other security tools for real-time monitoring and incident response across both centralized and distributed environments. This integration ensures seamless incorporation of DNS tunneling detection into existing security workflows. By addressing the integration gap found in many current models, the proposed solution will allow for faster, more coordinated responses to security incidents, enhancing overall visibility and improving incident management.

1.3.4. Lack of Adaptability to Evolving Threats

DNS tunneling techniques change quickly as attackers modify encoding, query patterns, and domain names to avoid detection [1]. Many solutions do not support rapid updates or retraining to handle new threats, which means they can quickly become obsolete or ineffective as attackers evolve their tactics. Adaptability is key to maintaining detection accuracy over time. The proposed solution fills this gap with continuous retraining and drift detection, ensuring the model remains effective against emerging tunneling strategies.

1.3.5. Normalization & Enrichment of DNS Data

A major challenge in DNS tunneling detection is the variability in DNS traffic. Raw DNS queries are often hard to interpret without proper normalization and enrichment. Normalization standardizes the data for analysis, while enrichment adds context, such as WHOIS data or blacklist checks. This additional context improves detection by offering more meaningful features to the model, leading to better accuracy. The proposed solution addresses this by incorporating a Normalization & Enrichment module that processes raw DNS data before feature extraction, significantly enhancing detection efficiency and accuracy.

1.3.6. Emphasis on Recall Rate in Real-Time Detection

While achieving high accuracy in DNS tunneling detection is important, recall rate is a crucial metric, particularly in real-time detection. Even with high accuracy, missing true positives can result in serious risks, such as data loss or ongoing exfiltration of sensitive information. In operational environments, the consequences of failing to detect a threat are often more severe than the impact of a false positive, as attackers may continue their activities undetected. By focusing on maximizing recall rate alongside accuracy, the proposed solution ensures that critical threats are detected, even if they generate some false positives. Striking the right balance between accuracy and recall is essential for effective DNS tunneling detection.

1.3.7. Rule Engine Customizability

The proposed solution features a rule engine that allows for human input when necessary, enabling the customization of detection and response rules. This flexibility

is crucial for adapting the system to varying network environments and security needs. The rule engine also supports dynamic updates to detection criteria and response actions, ensuring it can be tailored to the specific requirements of the organization. This ensures the DNS tunneling detection system is adaptable to address evolving threats and the unique challenges faced by different enterprises.

1.4. Research Problem

1.4.1. Research Question

How can a comprehensive, real-time DNS tunneling and data exfiltration detection system be designed that integrates both plaintext and encrypted DNS traffic analysis, while also addressing deployment flexibility, SIEM integration, and continuous adaptability to evolving attack techniques?

1.4.2. Approach

To address this question, this research will develop a multi-component detection framework that integrates real-time detection, metadata-based analysis, and machine learning models to identify DNS tunneling in both plaintext and encrypted DNS traffic (DoH/DoT), without requiring decryption. The system will prioritize seamless integration with enterprise security infrastructures, ensuring efficient alerting and incident response via SIEM systems. The research also emphasizes the need for a feedback loop to enable model retraining and drift detection, ensuring the system adapts to new tunneling techniques over time.

The approach combines advanced methods in DNS traffic analysis, machine learning, and network security, while tackling operational challenges like real-time alerting and scalability, ensuring the solution can be effectively deployed across different network environments.

Refer to Table 1, which explains the research gaps and demonstrates how the research problem is being addressed by the proposed solution.

Table 1: Comparative Research Gap in Tabular View

Study / Solution	Plain DNS Detection	DoH/DoT Handling	Real-Time Detection	Feature Extraction	Supervised ML/DL Models	Anomaly Detection	Hybrid Models	Decisioning Layer	Telemetry Storage	SIEM Integration	Threat Intel Enrichment
Liang et al. (FECC, CNN+Clustering)	Implemented	Not Implemented	Not Implemented	Implemented	Implemented (CNN + k-means)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Borges et al. (Cloud Platform, Statistical)	Implemented	Not Implemented	Not Implemented	Limited	Not Implemented	Implemented (Anomaly Detection)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Jiacheng Zhang et al. (Deep Learning Models)	Implemented	Not Implemented	Not Implemented	Implemented	Implemented (1D-CNN, RNN, DNN)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Sakarkar et al. (Deep Learning Packets)	Implemented	Not Implemented	Not Implemented	Implemented	Implemented (LSTM, CNN, RNN)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Qi et al. (Bigram Character Frequency)	Implemented	Not Implemented	Partially (Real-time scoring)	Implemented (Bigram Scoring)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Zhan et al. (DoH Exfiltration TLS FP)	Not Implemented	Implemented (DoH TLS FP)	Not Implemented	Limited	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Chen et al. (LSTM Covert Channels)	Implemented	Not Implemented	Not Implemented	Implemented	Implemented (LSTM)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented

Liu et al. (Behavior Features + ML)	Implemented	Not Implemented	Not Implemented	Implemented (Behavior Features)	Implemented (SVM, LR, DT)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Y. Chen & Li (Char-CNN, no FE)	Implemented	Not Implemented	Not Implemented	Not Implemented (End-to-end DL)	Implemented (Char-CNN)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Chowdhary et al. (Cache Miss + ML)	Implemented	Not Implemented	Not Implemented	Implemented (Entropy, Cache Miss)	Implemented (NB, SVM, RF, KNN)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
X.D. Li et al. (Campus ML dataset)	Implemented	Not Implemented	Not Implemented	Implemented	Implemented (LR, RF)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Y. Chen & Li (High Accuracy, No FE)	Implemented	Not Implemented	Not Implemented	Not Implemented (End-to-end DL)	Implemented (CNN/LSTM/GRU)	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Almusawi et al. (Multilabel SVM)	Implemented	Not Implemented	Not Implemented	Implemented	Implemented (Multilabel SVM)	Not Implemented	Implemented	Not Implemented	Not Implemented	Not Implemented	Not Implemented
Proposed Solution	Implemented	Implemented (No Decryption DoH/DoT)	Implemented (Near Real-Time Alerts)	Implemented (Statistical, Sliding Window, Seq)	Implemented	Implemented	Implemented	Implemented (Rule Engine + ML)	Implemented (NoSQL JSON logs)	Implemented (Enterprise SIEM)	Implemented (WHOIS, NXDomain, Allow/Deny)

2. OBJECTIVES

2.1.Main Objective

To design and develop a comprehensive real-time DNS tunneling and data exfiltration detection system that efficiently integrates both plaintext and encrypted DNS traffic analysis, ensuring seamless deployment in operational environments, while addressing flexibility, scalability, and continuous adaptability to evolving attack techniques.

2.2.Specific Objectives

1. **Literature Review** – Study existing DNS tunneling and encrypted DNS detection methods, noting their strengths, weaknesses, and gaps. Focus on the limitations related to real-time detection, encrypted DNS (DoH/DoT) without decryption, integration with security tools, and adaptability to evolving threats.
2. **Feature Engineering** – Create a unified set of traffic features for detecting both plaintext and encrypted DNS threats, leveraging metadata, statistical patterns, and behavioral analysis. Incorporate enriched context (e.g., WHOIS data, blacklist checks) to improve detection accuracy.
3. **Model Development** – Train and evaluate a hybrid AI/ML model for near-real-time DNS tunneling detection, combining supervised classification and anomaly detection to optimize accuracy, precision, recall, and latency. Incorporate continuous retraining and drift detection to adapt to new tunneling techniques.
4. **Framework Design** – Build a modular detection framework that integrates data capture, feature extraction, ML-based classification, and rule-based decision-making. Ensure real-time detection, flexibility, and adaptability to various operational environments.
5. **Custom Rule Engine** – Develop a rule engine to support dynamic input, enabling customizable detection and response rules specific to each deployment environment. Incorporate features like environment-specific policies, alert thresholds, and whitelisting/blacklisting.

- 6. Deployment Modes** – Enable both endpoint-based detection for individual devices during initial development testing and SIEM integration for enterprise-level monitoring, allowing the detection system to work seamlessly with existing security infrastructures and alerting systems.
- 7. Security and Performance Testing** – Assess the system’s resistance to evasion techniques, measure resource usage under real-time conditions, and evaluate the detection speed, ensuring low-latency performance in operational networks. Additionally, assess the security of the system itself, evaluating its resilience against potential attacks, vulnerabilities, and misuse during deployment and operational use.
- 8. Validation** – Test the system in a controlled lab environment with simulated normal and malicious DNS traffic, evaluating its accuracy, recall, precision, false positives, and latency to ensure effective threat detection under various scenarios.
- 9. Comparative Analysis** – Compare the system’s performance, adaptability, and operational efficiency with state-of-the-art DNS tunneling and exfiltration detection methods, emphasizing the system’s ability to detect encrypted DNS traffic without decryption and adapt to evolving attack techniques.
- 10. Scalability Assessment** – Evaluate the system's suitability for large-scale deployments in diverse network environments, ensuring that it can handle the high-volume traffic and integration challenges of enterprise environments.
- 11. Feature and Model Adaptation** – Develop a process for selecting the most effective detection features and updating the model to handle emerging tunneling techniques. Incorporate feedback loops and continuous model retraining to adapt to evolving DNS tunneling behaviors.
- 12. Documentation** – Produce comprehensive technical documentation and deployment guidelines, including detailed steps for real-world implementation, integration with existing security tools, system tuning for various network

environments, and AUP (Acceptable Use Policy) to ensure proper usage and compliance with organizational policies during deployment and testing.

3. METHODOLOGY

3.1. Methodology Description

3.1.1. Overall Approach & Success Criteria

A hybrid approach will be applied with:

1. Supervised classification – to differentiate between "good" and "bad" traffic based on labeled data, ensuring accurate identification of known threats.
2. **Anomaly detection** – to identify unusual or previously unseen patterns in DNS traffic, helping to detect new or evolving tunneling techniques.

The system is designed to operate on live network streams rather than relying solely on stored data.

The system is considered successful if:

1. A decision is made for each network flow within 200 milliseconds (for 95% of cases).

Detection latency (from feature extraction to verdict) is ≤ 200 milliseconds (for 95% of flows).

2. Malicious traffic is detected with at least 85% recall and 70% precision in lab validation, with PR-AUC as the primary metric.

Malicious recall is ≥ 0.95 with precision ≥ 0.90 in lab validation, with PR-AUC as the primary evaluation metric.

3. False positives are kept to fewer than one per host per day in steady state, with tunable thresholds via the rule engine.

False positives are kept ≤ 1 per host per day in steady-state conditions, with tunable thresholds via the rule engine.

4. The system's resource consumption on endpoints does not exceed 5% CPU usage and 150 MB RAM.

3.1.2. System Architecture

The system architecture follows a modular design to efficiently handle DNS tunneling detection in both plaintext and encrypted DNS traffic (DoH/DoT). It consists of the following key components:

1. DNS Capture Module

The system begins with the DNS Capture Module, responsible for collecting both plaintext DNS and encrypted DNS traffic (DoH/DoT). This module gathers raw DNS data from network traffic, providing the initial input for the system.

2. Normalization & Enrichment Module

After raw data is captured, it is passed to the Normalization & Enrichment Module. This module parses and standardizes the raw data into a consistent format (e.g., 5-tuple, timestamps, query types, TTL, etc.). Additionally, the data is enriched with contextual information such as WHOIS data, domain age, NXDomain rates, and allow/deny lists. This enriched data is then sent to the next stage for further processing. Also the metadata required to analyze the encrypted DNS traffic will be extracted within this module.

3. Feature Extraction Module

In this step, the normalized and enriched data is processed in the Feature Extraction Module. The system uses statistical and sequence features extracted through sliding window processing to create a rich set of features for analysis. These features are then forwarded to the Model Inference Module for detection.

4. Model Inference Module

The Model Inference Module applies a hybrid approach to DNS tunneling detection, incorporating supervised classification (*via XGBoost*) *and* anomaly detection (*via Isolation Forest*). This combination enables the system to effectively detect both known and novel tunneling patterns in real-time, adapting

dynamically to evolving attack techniques. The module generates predictions and inference results based on the processed features, which are then passed to the next component for decision-making.

5. Decisioning Layer

The output from the Model Inference Module is processed by the Decisioning Layer. Here, decisions are made regarding how to respond to detected threats (e.g., allow, alert, rate-limit, block, or quarantine). A key feature of this layer is the **Rule Engine**, which allows for human input and customization. This enables dynamic decision-making based on organizational policies, ensuring the system can adapt to specific network environments and security needs. The output of the decision layer is simultaneously sent to the Telemetry Storage Module and the Feedback Loop for further processing.

6. Telemetry Storage Module (NoSQL DB)

The Telemetry Storage Module is a NoSQL database where decisioning metadata (such as detection results, timestamps, and feature values) is stored in JSON format. This data is crucial for post-event analysis, reporting, and auditing. The telemetry data can also be used to track system performance and trends over time. The metadata stored in telemetry storage will be forwarded to both the SIEM system and the alerting module (endpoint detection during development testing phases) simultaneously for further analysis and response.

7. Feedback Loop

The Feedback Loop is responsible for continuous improvement of the system. It handles event labeling, and drift detection. The labeled events are then forward to the supervised model for retraining ensuring the system adapts to new attack vectors. The retrained model and performance feedback are fed back into the Model Inference Module to improve its accuracy and responsiveness.

This modular and scalable architecture ensures that the system is capable of handling both live network traffic and large-scale deployments, providing a robust and adaptive solution for DNS tunneling detection across various environments. Refer to **Figure 2:**

The system diagram for a detailed visualization of the system components and their interactions.

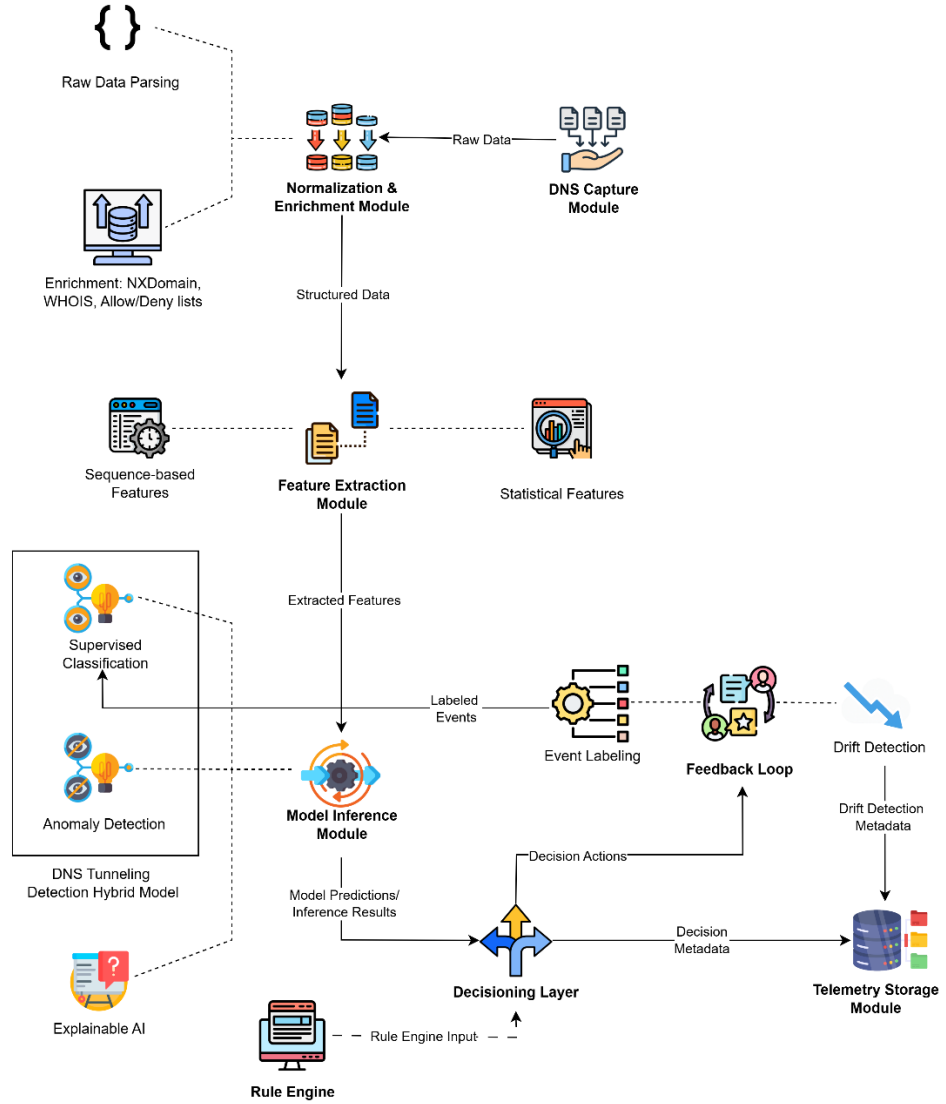


Figure 2: System Architecture Diagram

3.1.3. Data Collection & Labeling

To develop an effective near real-time DNS tunneling and data exfiltration detection system, a robust data collection and labeling strategy is essential. This section outlines the data sources, methods for data collection, and labeling strategies to ensure high-quality annotations for training the AI/ML model.

1. Data Sources

The following publicly available datasets will be used in the early stages of model development, while custom data collection methods will be implemented as the project progresses.

DNS Exfiltration Datasets:

- Mendeley DNS Exfiltration Dataset

Over 50 million anonymized DNS requests from an ISP's DNS server, including both benign and malicious traffic. Useful for detecting DNS exfiltration behaviors.

- Kaggle DNS Tunneling Dataset

1.58 million DNS queries with 797,626 benign and 785,537 malicious samples, generated using nine DNS exfiltration tools.

- CIC-Bell-DNS-EXF-2021

A dataset with 270.8 MB of DNS traffic, including 323,698 heavy attack samples and 641,642 benign samples, ideal for real-world testing.

DoH/DoT Tunneling Datasets:

- DoH-Tunnel-Traffic-HKD

PCAP files of DoH tunnel traffic with extracted statistical features. Useful for training models on DoH tunneling detection.

- CIRA-CIC-DoHBrw-2020

A dataset focused on DoH traffic for covert channels and tunnels, containing both benign and malicious traffic.

2. Data Collection Methods

Real-Time Network Traffic Capture:

DNS traffic will be collected using tools like Wireshark and tcpdump for both plaintext and encrypted DNS (DoH/DoT) traffic, which will provide real-world validation and data for retraining.

Synthetic Data Generation:

To supplement the datasets, synthetic DNS traffic can be generated using tools like dnsgen and DNSExfiltrator, which will allow for custom datasets tailored to specific attack scenarios.

3. Data Labeling Strategy

Data labeling will use a combination of supervised and unsupervised learning strategies to handle both known and unknown attack scenarios.

Supervised Classification

Supervised classification will be used to distinguish between known malicious and benign traffic by utilizing labeled data.

Labeling Strategy:

- **Manual Labeling of Malicious Traffic:** Traffic originating from known DNS exfiltration tools (e.g., dnscat2, iodine) will be marked as malicious.

Heuristic Labeling: Malicious traffic will be identified through heuristic patterns, such as unusually long domain names or high query frequencies.

Anomaly Detection

The focus of anomaly detection will be to identify abnormal patterns in DNS traffic by first defining what normal behavior looks like and then spotting any deviations from it.

Labeling Strategy:

- **Normal Traffic Labeling (Supervised Initial Training):** A portion of normal traffic will be manually marked as “benign” to establish the baseline.
- **Synthetic Attack Data Generation:** Tools such as DNSExfiltrator will generate rare attack patterns, which will be labeled as “anomalous.”

- **Clustering & Self-Labeling:** Unsupervised clustering methods will categorize traffic into normal and anomalous groups, with human annotators validating these groupings.

Hybrid Labeling Strategy

A hybrid labeling approach will combine the strengths of these two strategies:

- **Supervised Classification** will identify known threats based on labeled data.
- **Anomaly Detection** will define normal traffic and detect new attack patterns.

Ground Truth Data

The following resources will assist in generating accurate ground truth for malicious traffic:

- **Threat Intelligence Feeds:** Public and commercial feeds for identifying malicious domains and attack patterns.
- **Domain Reputation Services:** Use reputation systems to check and label malicious domains used in DNS tunneling and exfiltration.

4. Dataset Splits and Evaluation

To train, validate, and test the model, the dataset will be split as follows:

- **Training Set:** 70-80% of the data will be used for training the model, including a mix of benign and malicious traffic.
- **Validation Set:** 10-15% will be used for hyperparameter tuning and model validation.
- **Test Set:** 10-15% will be reserved for final testing, ensuring the model's generalizability.

This streamlined approach to data collection and labeling ensures the DNS tunneling and data exfiltration detection system is trained with high-quality, accurate data, enabling it to handle both known and emerging attack techniques effectively.

3.1.4. Feature Engineering

1. Real-Time Statistical Features

These features are derived from DNS traffic statistics to identify unusual patterns as they emerge. Examples include query length, query frequency within defined time intervals, DNS packet sizes, time between consecutive queries, and the average Time to Live (TTL) of responses.

2. Real-Time Domain Name Features

This set of features examines the characteristics of domain names in DNS traffic, since malicious activities often manipulate domain names. Examples include domain name length, entropy (randomness) of domain names, frequency of certain domains over time, and syntactic features such as the number of subdomains or special characters in domain names.

3. Temporal Features

These features track the timing of DNS queries and responses, helping to identify patterns that could indicate tunneling or data exfiltration. Examples include the time of day queries are made, bursts of high-frequency queries, intervals between queries, and trend aggregation using sliding time windows.

4. Traffic Volume Features

These features measure the total volume of DNS traffic over time, useful for identifying large-scale exfiltration activities. Examples include query volume per time window, total data transferred via DNS, and the number of DNS requests made by clients in real time.

5. DNS Response Features

These features focus on DNS responses, as they may point to anomalous or malicious activity. Examples include response code distribution (e.g., NOERROR, NXDOMAIN), DNS response sizes, number of records in responses, and variations in response times.

6. Machine Learning Features (Derived from Raw DNS Data)

These features involve extracting higher-level patterns from raw DNS data using machine learning techniques. Examples include principal component analysis (PCA) for dimensionality reduction, reconstruction errors from autoencoders to detect deviations, and clustering labels from methods like K-Means or DBSCAN.

7. DoH/DoT-Specific Features

These features focus on encrypted DNS traffic (DoH/DoT) and aim to detect tunneling activities. Examples include TLS fingerprints for identifying session specifics, payload sizes in encrypted DNS traffic, TLS session durations, and query encoding methods in DoH/DoT traffic.

8. Graph-Based Features

Graph-based features analyze the relationships between DNS queries, responses, and domain names in a graph structure. Examples include traffic flow graphs with nodes representing clients and servers, and co-occurrence matrices for tracking domains that often appear together in queries.

9. Real-Time Aggregation and Rollups

These features summarize DNS data over short time windows to detect attack-related trends. Examples include rolling aggregates of packet sizes, time-windowed DNS query counts, and moving averages of query intervals to monitor ongoing behavior.

10. Real-Time Traffic Flow Features

These features track DNS traffic flow between clients and servers. Examples include the time between DNS queries and responses, volume of connections to specific DNS servers, and the detection of co-occurring queries where certain queries tend to appear together.

11. Anomaly Detection Features

Anomaly detection features use machine learning models to identify deviations from typical DNS traffic patterns. Examples include anomaly scores from models like Isolation Forest or Autoencoders, detecting statistical outliers in query

patterns, and using real-time anomaly detection with a sliding window approach to flag suspicious activity dynamically.

3.1.5. Model Development

The Model Development process focuses on building an effective machine learning model for DNS tunneling and data exfiltration detection using a hybrid approach combining anomaly detection, supervised classification, and real-time optimization.

1. Model Training

- Supervised Classification (XGBoost): Classifies DNS traffic as benign or malicious based on labeled data, detecting known tunneling patterns.
- Anomaly Detection (Isolation Forest): Identifies novel or previously unseen traffic patterns by learning the normal behavior of DNS traffic and flagging deviations as anomalies.

2. Model Explainability

- SHAP values will be used to explain model decisions, ensuring transparency and helping analysts understand why certain traffic is flagged.

3. Evaluation Metrics

- Precision, Recall, F1-Score, and PR-AUC will be used to evaluate performance, with latency ($\leq 200\text{ms}$ for 95% of queries) and throughput (queries per second) ensuring real-time operation.

4. Model Retraining and Adaptability

- A feedback loop will allow continuous retraining based on new data and emerging attack techniques, ensuring the model adapts to evolving threats over time.

4.1.6. Deployment Modes

1. Endpoint Mode

- A lightweight agent deployed on individual devices to collect and process DNS traffic locally.

- Alerts and decisions are generated on the endpoint itself and can be forwarded to centralized systems for further analysis.
- Ideal for early testing and small-scale environments.

2. SIEM-Integrated Mode

- Centralized collection of DNS traffic from network devices or endpoint agents.
- Processes data for feature extraction, inference, and decision-making, with logs forwarded to SIEM systems (e.g., ELK, Graylog, Splunk) for monitoring.
- Suitable for enterprise-level deployments requiring centralized monitoring and integration with existing security infrastructure.

3. Hybrid Deployment Mode

- Combines Endpoint Mode and SIEM-Integrated Mode for flexibility, with local detection on endpoints and centralized monitoring through SIEM.
- Useful for distributed environments that need both local and centralized monitoring.

For both deployment modes, the same model inference, decision layer, and all other components will be used, except for the alerting section.

4.1.7. Security, Evasion Resistance & Performance Testing

1. Adversarial/Evasion Tests

The system will undergo various evasion tests to assess its ability to detect sophisticated DNS tunneling techniques, including:

- Encoding variations: Test the system's resistance to encoding schemes that obfuscate tunneling data.
- Subdomain depth manipulation: Evaluate detection performance when the depth of subdomains is altered.
- Query type (QTYPE) camouflage: Assess if the system can detect tunneling attempts hidden behind unusual or uncommon DNS query types.

- TLS manipulation: Test the system's ability to detect malicious traffic even when using modified TLS fingerprints or other evasion tactics.

2. Performance Profiling

- The system's latency and resource usage will be measured under different load conditions:
- Latency: The detection time for processing DNS queries will be assessed to ensure real-time operation (i.e., ≤ 200 milliseconds for 95% of queries).
- Resource Usage: The system's CPU and memory usage will be monitored to ensure it remains within acceptable limits (e.g., $\leq 5\%$ CPU and ≤ 150 MB RAM for endpoint deployments).

3. Robustness Testing

- Chaos tests will be conducted to simulate network instability and evaluate the system's resilience under conditions such as:
- Packet loss: Test system performance when packets are lost during transmission.
- Out-of-order arrivals: Assess how well the system can handle DNS traffic where packet order is disrupted.

4. Scalability Testing

- The system's ability to scale across large network environments will be tested, ensuring that it can handle increasing volumes of DNS traffic without degrading performance.
- Horizontal scaling will be tested by simulating a larger number of DNS queries from multiple endpoints or network sources.

4.1.8. Validation (Lab)

1. Testbed Design

A testbed will be set up using a mix of benign enterprise traffic, synthetic malicious traffic (e.g., generated using tools like iodine, dnscat2), and custom-

generated malicious queries to simulate various attack scenarios. These queries will be tailored to mimic DNS tunneling, data exfiltration, and other suspicious behaviors, providing a comprehensive validation environment.

2. Measurements

The system's performance will be evaluated using the following key metrics:

- Accuracy: Percentage of correctly classified traffic (both benign and malicious).
- Precision and Recall: Assessing the system's ability to detect true positives while minimizing false positives.
- F1-Score: Balancing precision and recall to ensure reliable detection.
- PR-AUC (Precision-Recall Area Under Curve): Focused on handling class imbalance and detecting rare malicious events.
- Latency: Ensuring detection occurs in real-time, with a target of $\leq 200\text{ms}$ for 95% of queries.
- False Positive Rate: Monitoring the number of benign queries flagged as malicious to ensure minimal disruption.

3. Ablation Testing

- Ablation studies will be performed to assess the importance of different feature sets and model variants in the detection process.
- By selectively removing or modifying specific features or models, the system's performance will be evaluated to understand which factors most contribute to the detection accuracy.

4. Lab Validation

- The system will undergo lab-based validation in a controlled environment using both pre-recorded traffic and real-time simulations to ensure the models and components are functioning correctly.

- The lab environment will allow for fine-tuning the system and ensuring that detection thresholds are properly set before full deployment.

4.1.9. Scalability Assessment

1. Horizontal Scaling

- The system will be evaluated for its ability to scale horizontally, handling increasing volumes of DNS traffic across multiple devices or network segments.
- This scalability assessment will ensure the system performs efficiently in large-scale environments, handling thousands or even millions of DNS queries without degradation in performance.

2. Back-pressure Strategy

- A back-pressure strategy will be implemented to manage high traffic volumes effectively, preventing the system from becoming overwhelmed.

3. Storage Scalability

- The system's storage capabilities will be assessed by simulating large-scale DNS traffic and measuring how well the telemetry and decision logs are stored in NoSQL databases.
- The system will be evaluated for its ability to handle long-term data retention, with an emphasis on efficient querying, indexing, and cost optimization when scaling storage across multiple devices and networks.

4.1.10. Documentation & Deliverables

The following documentation and deliverables will be provided:

- Architecture Document: System architecture overview aligned with Architecture Diagram.
- Feature Dictionary: A list of features with definitions and tests.
- Data Handling SOPs: Guidelines for data collection, labeling, enrichment, and storage.
- Model Card: Information on training data, model metrics, limitations, and fairness notes.
- Deployment Guide: Instructions for deployment in endpoint and SIEM modes.
- SIEM Content Pack: Dashboards and alerts for integration with SIEM systems.
- Runbooks: Guides for tuning thresholds, managing allowlists/blocklists, and retraining.
- Evaluation Report: Performance evaluation with reproducible notebooks/pcaps.
- AUP (Acceptable Use Policy): Guidelines for acceptable system usage and data handling.

4.2. Tools & Technologies

Table 2: Tools & Technologies

Tool/Technology	Purpose
Python	The primary programming language for developing the system, used for implementing machine learning models, network traffic capture, and analysis.
Wireshark, tcpdump, Zeek, Suricata	Tools for capturing DNS traffic (both plaintext and encrypted, including DoH/DoT) and TLS metadata (SNI, JA3 fingerprints) for analysis and monitoring.
dnsgen, DNSExfiltrator	Tools for generating both benign and malicious DNS traffic, simulating DNS tunneling and exfiltration attacks to create custom datasets for model training.

XGBoost	Machine learning model used for supervised classification, distinguishing between benign and malicious DNS traffic.
Isolation Forest	Anomaly detection technique used to identify novel or unseen traffic patterns by learning normal DNS traffic behavior.
SHAP	Used for model explainability, helping to interpret XGBoost feature importance and decision-making in detecting DNS tunneling.
NoSQL Database	Stores telemetry data (detection results, features, etc.) in a flexible and scalable format, supporting large-scale deployments.
Wazuh	Open-source SIEM platform used to monitor and correlate security events from the DNS threat detection system.

4.3. Anticipated Conclusion

This project will develop a real-time DNS tunneling and data exfiltration detection system using AI/ML techniques to analyze both plaintext and encrypted DNS traffic. By combining supervised classification and anomaly detection, the system will detect known and emerging threats with high accuracy.

The system's integration with SIEM platforms and customizable rule engine will allow it to be deployed in both small-scale and large-scale environments. The emphasis on real-time detection, low-latency performance, and efficient resource usage ensures that it can operate effectively in operational networks.

Ultimately, this solution addresses a key gap in DNS traffic monitoring, providing a scalable and adaptable tool to enhance network security and improve response times to malicious activities.

4.4. Work Breakdown Structure

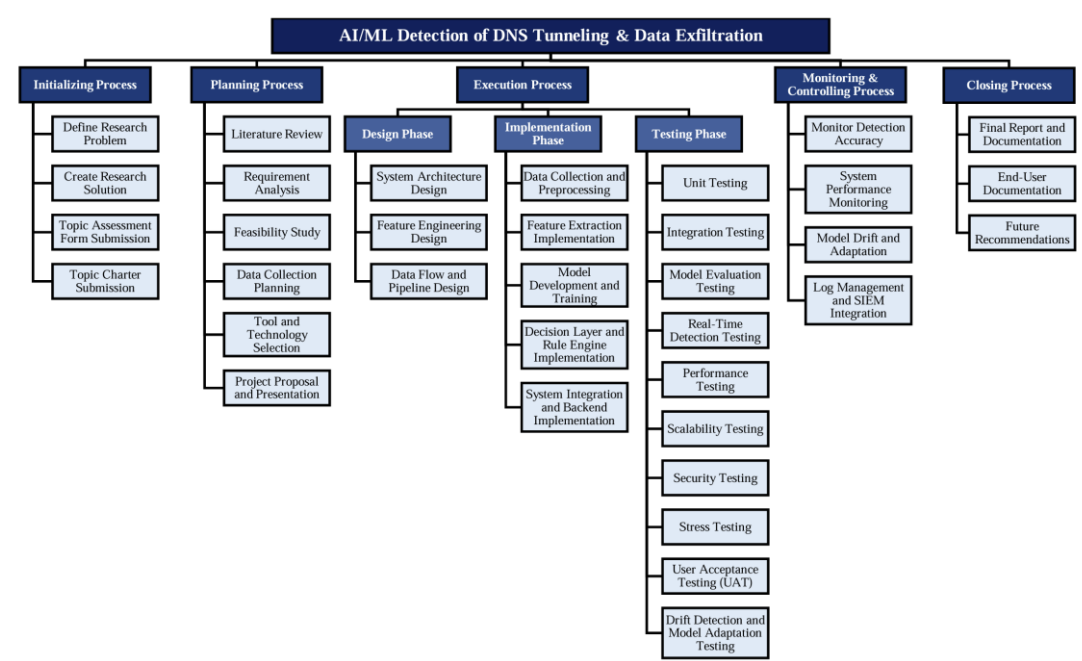


Figure 3: Work Breakdown Structure

4.5. Project Gantt Chart

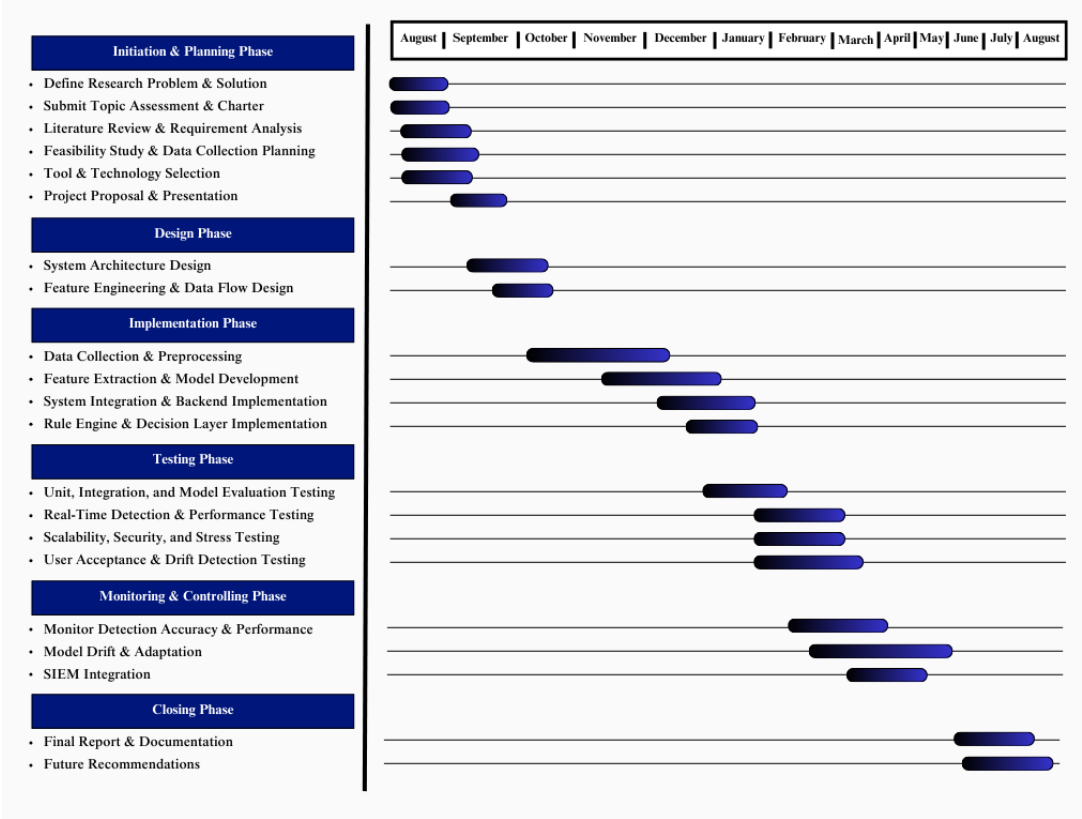


Figure 4: Project Gantt Chart

5. PROJECT REQUIREMENTS

5.1. Functional Requirements

- Real-Time DNS Threat Detection: Detect DNS tunneling and data exfiltration in near real-time ($\leq 200\text{ms}$ for 95% of queries).
- Traffic Capture & Analysis: Capture and analyze both plaintext and encrypted DNS (DoH/DoT) traffic, with enrichment (e.g., WHOIS data).
- Machine Learning Integration: Use a hybrid approach combining anomaly detection and supervised classification.
- Customizable Rule Engine: Allow users to define detection and response rules specific to their network.
- SIEM Integration: Forward logs and alerts to SIEM systems for centralized monitoring and incident response.
- Telemetry & Feedback Loop: Store metadata and enable model retraining to adapt to new threats.
- Threat Intelligence Enrichment: Integrate threat intelligence feeds for enhanced detection.

5.2. Non-Functional Requirements

- Scalability: Handle high DNS traffic volumes with horizontal scaling.
- Performance: Ensure $\leq 200\text{ms}$ detection latency, low resource usage, and maintain high recall and precision ($\geq 85\%$ recall, $\geq 70\%$ precision) in detecting DNS tunneling and data exfiltration.
- Reliability: Provide fault tolerance and uninterrupted monitoring.
- Security: Detect evasion tactics, analyze encrypted DNS without decryption, and ensure the product itself is secured while preserving the confidentiality, integrity, and availability (CIA) of data using IAAA (Identity, Authentication, Authorization, and Auditing) controls.

- Usability: Ensure user-friendly customization and integration with security infrastructures.
- Maintainability: Support model updates and retraining with minimal downtime.

5.3. User Requirements

- Endpoint Deployment: Deploy lightweight agents for local DNS traffic analysis in small environments.
- Centralized Monitoring (SIEM Mode): Collect and analyze traffic centrally, integrating with SIEM systems.
- Real-Time Alerts & Response: Generate alerts and actionable responses for suspicious DNS traffic.
- Adaptability: Customize detection thresholds and response actions based on network policies.
- Custom Reporting: Generate detailed reports for analysis and incident tracking.
- Ease of Integration: Seamlessly integrate with existing security systems.

5.4. Expected test cases

1. **Test Case:** Detection of DNS tunneling traffic

Expected Result: The system identifies and flags malicious DNS tunneling traffic within the required latency threshold ($\leq 200\text{ms}$ for 95% of queries).

2. **Test Case:** Handling of encrypted DNS (DoH/DoT) traffic

Expected Result: The system correctly detects suspicious patterns in encrypted DNS traffic without decryption and generates a real-time alert.

REFERENCES

- [1] G. Gürsoy, A. Varol and A. Nasab, "DNS Tunnel Problem In Cybersecurity," in *12th International Symposium on Digital Forensics and Security (ISDFS 2024)*, 2024.
- [2] J. Liang, S. Wang, S. Zhao and S. Chen, "FECC: DNS tunnel detection model based on CNN and clustering," *Computers & Security*, vol. 128, 2023.
- [3] L. d. S. B. Borges, R. d. O. Albuquerque and R. T. d. S. Júnior, "A security model for DNS tunnel detection on cloud platform," in *2022 Workshop on Communication Networks and Power Systems (WCNPS)*, Fortaleza, Brazil, 2022.
- [4] J. Zhang, L. Yang, S. Yu and J. Ma, "A DNS Tunneling Detection Method Based on Deep Learning Models to Prevent Data Exfiltration," in *Network and System Security (NSS 2019)*, Cham, 2019.
- [5] M. Zhan, Y. Li, G. Yu, B. Li and W. Wang, "Detecting DNS over HTTPS based data exfiltration," *Computer Networks*, vol. 209, 2022.
- [6] C. Qi, X. Chen, C. Xu, J. Shi and P. Liu, "A Bigram based Real Time DNS Tunnel Detection Approach," in *Procedia Computer Science*, 852-860.
- [7] G. Sakarkar, M. K. H. Kolekar, K. Paithankar, G. Patil, P. Dutta, R. Chaturvedi and S. Kumar, "Advance Approach for Detection of DNS Tunneling Attack from Network Packets Using Deep Learning Algorithms," *Advances in Distributed Computing and Artificial Intelligence Journal*, vol. 10, pp. 241-266, 2021.
- [8] S. Chen, B. Lang, H. Liu, D. Li and C. Gao, "DNS covert channel detection method using the LSTM model," *Computers & Security*, vol. 104, 2021.
- [9] J. Liu, S. Li, Y. Zhang, J. Xiao, P. Chang and C. Peng, "Detecting DNS Tunnel through Binary-Classification Based on Behavior Features," in *IEEE Trustcom/BigDataSE/ICSS*, Sydney, NSW, Australia, 2017.
- [10] A. Chowdhary, M. Bhowmik and B. Rudra, "DNS Tunneling Detection using Machine Learning and Cache Miss Properties," in *Publication Name: 5th International Conference on Intelligent Computing and Control Systems (ICICCS)*, Madurai, India, 2021.

- [11] A. Ahmed and A. Haleh, "DNS Tunneling Detection Method Based on Multilabel Support Vector Machine," *Security and Communication Networks*, pp. 1-9, 2018.
- [12] Y. Chen and X. Li, "A High Accuracy DNS Tunnel Detection Method Without Feature Engineering," in *16th International Conference on Computational Intelligence and Security (CIS)*, Guangxi, China, 2020.
- [13] T. Nguyen and M. Park, "DoH Tunneling Detection System for Enterprise Network Using Deep Learning Technique," *Applied Sciences*, 2022.
- [14] Z. Jiacheng, Y. Li, Y. Shui and M. Jianfeng, "A DNS Tunneling Detection Method Based on Deep Learning Models to Prevent Data Exfiltration," in *Network and System Security (NSS 2019)*, 2019.